Modern Veri Biliminde Linux ve Unix Ekosistemi

Büyük Veri, MLOps, Yapay Zeka ve Süper Bilgisayar Mimarilerinin Çekirdek İşletim Sistemi

TARİHSEL MİMARİ

Unix Mimarisi ve Linux'un Doğuşu

UNIX Kökleri (1969)

Sahipli ve ticari sistemlerin öncüsü.

- Oracle Solaris
- IBM AIX & HP-UX
- Apple macOS (Darwin Çekirdeği)

LINUX Dönüşümü (1991)

Özgür, açık kaynaklı ve modüler çekirdek.

- POSIX Standartları Uyumlu
- GPL Lisanslı Açık Kaynak
- Küresel Topluluk Desteği

VERİ BİLİMİ ODAĞI

Veri Bilimciler İçin Linux Neden Zorunlu?

GPU ve CUDA Entegrasyonu

NVIDIA CUDA sürücüleri en yüksek sürücü verimliliğini Linux çekirdeğinde sunar.

Yüksek Bellek Yönetimi

Büyük veri kümelerinde RAM ve swap alanını doğrudan kontrol imkanı sağlar.

Açık Kaynak Yığını

PyTorch, TensorFlow ve C++ kütüphaneleri ilk olarak Linux üzerinde geliştirilir.

GÜNCEL VERİ / SÜPER BİLGİSAYARLAR

Küresel Yapay Zeka Altyapısında Linux Dominesi

%100

TOP500 Dünyanın En Hızlı Süper Bilgisayarları

Dünyanın en güçlü 500 süper bilgisayarının tamamı Linux işletim sistemi üzerinde büyük dil modelleri ve veri analizi koşturmaktadır.

MİMARİ VE BİLEŞENLER

Linux İşletim Sistemi Anatomisi

1. Çekirdek (Kernel)
Donanım ile yazılım arasındaki kaynak yönetim katmanı.

2. Kabuk (Shell: Bash / Zsh)
Veri işleme ve otomasyon komutlarının yürütüldüğü arayüz.

3. Sistem Araçları ve GUI
Paket yöneticileri, masaüstü ortamları ve veri sürücüleri.



DAĞITIM REHBERİ — 1

Kurumsal ve Sunucu Odaklı Dağıtımlar

KÖK DAĞITIM

Debian

Yüksek kararlılık standartı.

EN POPÜLER VERİ STANDARDI

Ubuntu LTS

Geniş kütüphane ve GPU desteği.

KURUMSAL SUNUCU

RHEL / Rocky Linux

Kurumsal veri merkezleri altyapısı.

Geliştirici ve Özelleştirilmiş Dağıtımlar

Fedora Workstation

En yenilikçi paketler ve güncel Linux çekirdeği entegrasyonu.

openSUSE / SLE

YaST yönetim aracı ve güçlü kurumsal yapılandırma yeteneği.

Arch Linux

Özelleştirilebilir yapıda sürüm modeli ve tam modüler kontrol.

PRATİK UYGULAMA

Linux Terminali ile Veri Manipülasyonu

```
# Veri setinden hızlı özet çıkarma
cat dataset.csv | grep "ERROR" | awk -F',' '{print $3}' | sort | uniq -c

# GPU durumunu izleme
watch -n 1 nvidia-smi
```

Aşırı Hızlı Veri Ön İşleme

Grep, Sed ve Awk araçları gigabaytlarca veriyi Python'dan daha hızlı filtreler.

Uzaktan Sunucu Yönetimi

SSH ve Tmux ile GPU küme sunucularında kesintisiz model eğitimi.

MODERN VERİ ALTYAPISI

Konteyner Mimarileri ve MLOps



Docker & İzolasyon

Model bağımlılıklarını izole ederek her ortamda aynı sonuçları alma imkanı.

Kubernetes Orkestrasyonu

Dağıtık mikroservisler ve ölçeklenebilir yapay zeka servisi yaygınlaştırma.

ÖĞRENCİ GELİŞTİRME ORTAMI

Windows Üzerinde Hibrit Linux (WSL2)

WSL2 Çekirdeği

Windows içerisinde doğrudan yerel Linux çekirdeği çalıştırma imkanı.

VS Code Entegrasyonu

Linux ortamındaki dosyaları doğrudan Windows VS Code ile geliştirme.

Jupyter Lab & Conda

Izole Python sanal ortamları ve interaktif veri analizi defterleri.

SÜREÇ MİMARİSİ

Linux Üzerinde Veri Bilimi Yaşam Döngüsü

AŞAMA 01

Veri Toplama

Bash betikleri ve Wget / Curl ile otomatik veri aktarımı.

AŞAMA 02

Ön İşleme

Sed, Awk ve CSVKit ile yüksek hızlı veri temizleme.

AŞAMA 03

Model Eğitimi

CUDA ve PyTorch ile Paralel GPU hesaplama.

AŞAMA 04

Dağıtım (MLOps)

Docker konteynerleri ile canlıya alma.

LINUX TEMELLERİ

Büyük Resim: Kernel, Süreçler, Bellek ve User Space

Öğr. Gör. Dr. Uğur ÖZBEK

SAKARYA ÜNİVERSİTESİ

Linux’u anlamanın anahtarı: Soyutlama

Karmaşıklığı yok etmek değil, doğru ayrıntı düzeyinde ele almak

Karmaşık sistem

Aynı anda çalışan süreçler, servisler, bellek, disk ve ağ bileşenleri tek bir bütün oluşturur.

Abstraction

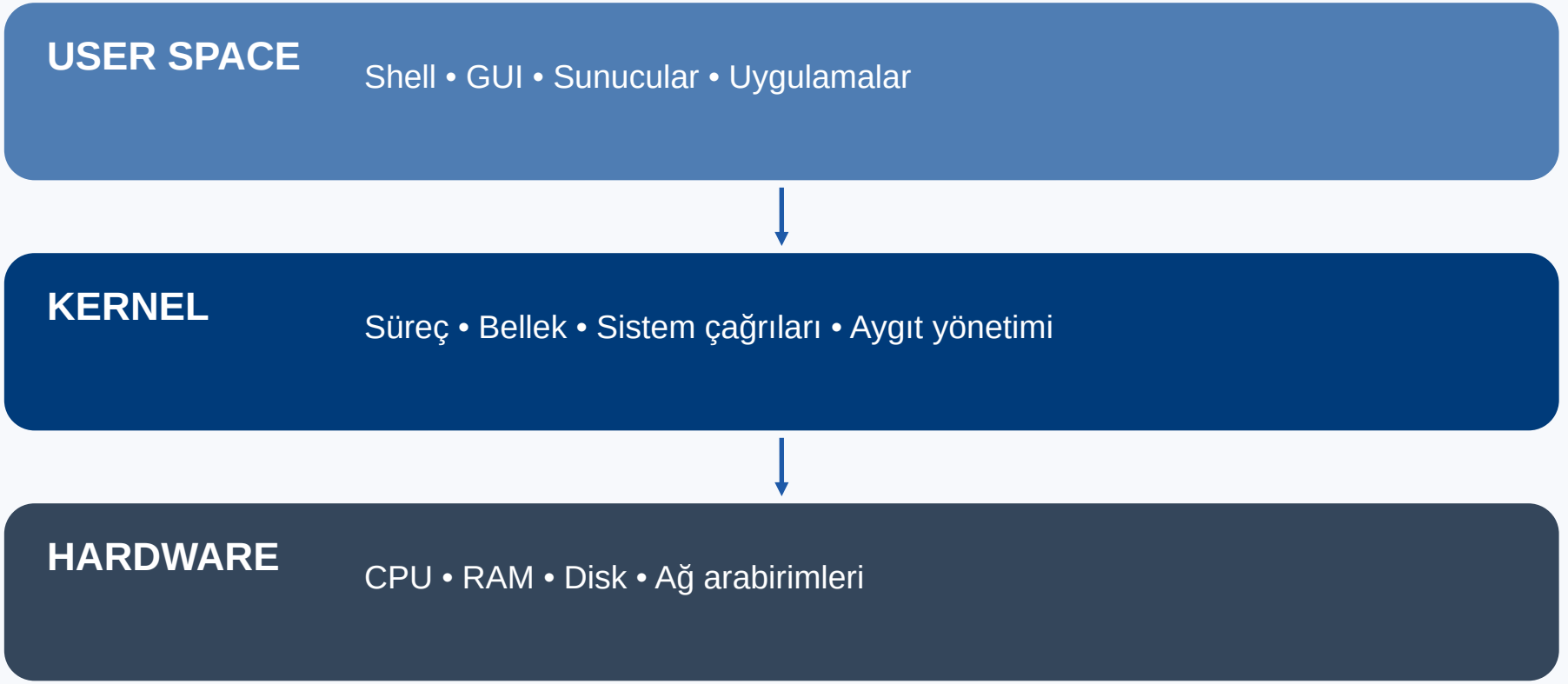
Bir bileşenin iç ayrıntılarını geçici olarak gizleyip temel işlevine ve arayüzüne odaklanır.

Problem çözme

Arızayı önce doğru katmana indirgemek, ardından gerekli ayrıntı düzeyine inmek mümkün olur.

Büyük resmi kur → doğru katmanı seç → gerektiği kadar derine in

Linux sisteminin üç temel düzeyi



Uygulamalar kaynakları doğrudan yönetmez; kernel aracılığıyla kullanır.

Ana bellek ve “state” kavramı

Çalışan sistemin aktif durumu bellekte temsil edilir

Main Memory (RAM)

Çalışan kernel ve süreçlerin aktif kod ve verileri bellekte bulunur.

CPU komutları ve verileri bellekten alır; sonuçları yine belleğe yazabilir.

State (Durum)

Belirli bir andaki çalışma durumunu temsil eden bilgidir.

Gerçek sistemlerde tek tek bitler yerine anlamlı durum tanımları kullanılır.

Neden önemli?

Bir süreç durdurulup daha sonra devam ettirilecekse gerekli çalışma durumunun korunması gerekir.

CPU ⇄ Main Memory

Kernel neyi yönetir?

Processes

CPU’yu hangi sürecin ve ne zaman kullanacağını yönetir.

Memory

Hangi belleğin kime ayrıldığını ve hangi alanların korunacağını takip eder.

Device Drivers

Donanım ile süreçler arasındaki donanıma özgü iletişimi sağlar.

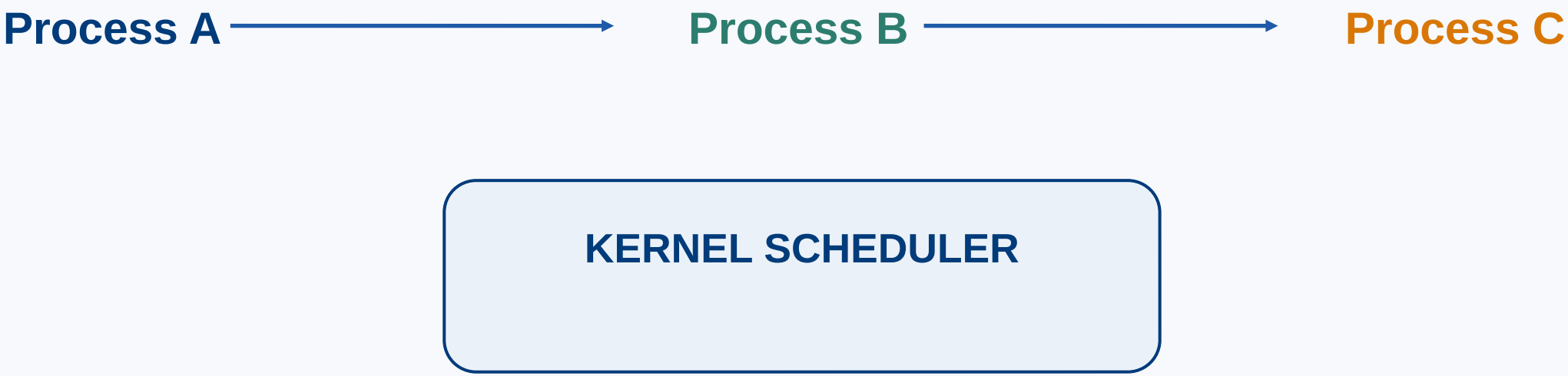
System Calls

User process’lerin kernel hizmetlerine kontrollü erişimini sağlar.

Bu dört alan ilerleyen konularda ayrı ayrı derinleştirilecektir.

Process Management

Çok sayıda süreç, sınırlı CPU zamanı



Hangi süreç CPU’yu ne zaman kullanacak?

Time Slice ve Multitasking

Tek çekirdekte eşzamanlılık hissi nasıl oluşur?



Time Slice

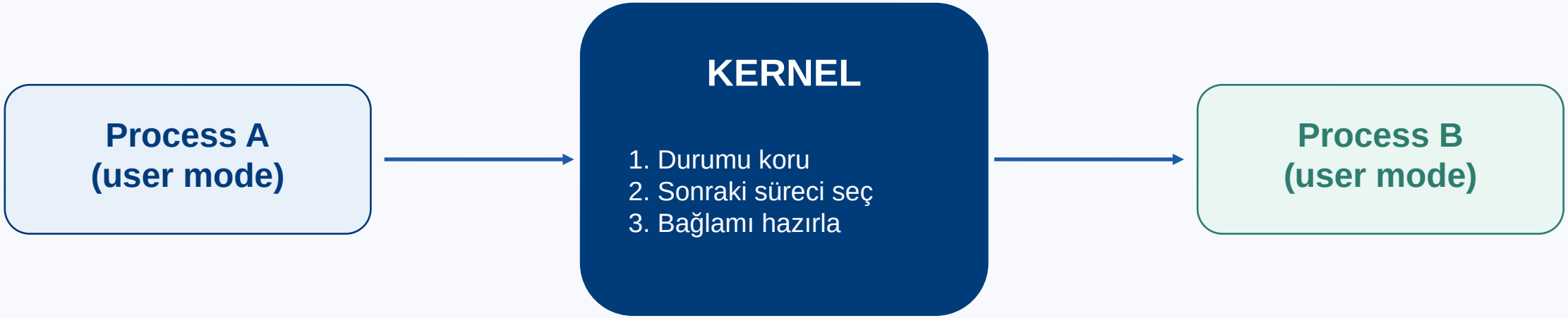
Bir sürecin CPU üzerinde çalışabildiği kısa zaman aralığı.

Multitasking

Süreçlerin kısa aralıklarla CPU'yu paylaşarak birlikte ilerlemesi.

Context Switch

CPU kontrolünün bir süreçten diğerine aktarılması



State + scheduling + kontrollü geçiş

Concurrency ve Parallelism

Aynı zaman aralığında ilerlemek ≠ fiziksel olarak aynı anda yürütülmek

Concurrency — Eşzamanlı ilerleme

Tek çekirdekte süreçler context switching ile aynı zaman aralığında ilerleyebilir.

A → B → C → A → …

Parallelism — Paralellik

Birden fazla CPU çekirdeğinde birden fazla süreç gerçekten aynı anda yürütülebilir.

Core 1 → A
Core 2 → B

Çok çekirdekli sistemlerde de scheduling ve context switching kullanılmaya devam eder.

Memory Management

Bellek yönetimi yalnızca “boş RAM bulmak” değildir

- Kernel için korumalı bellek alanı
- Her süreç için ayrı bellek alanı
- Süreçler arasında izolasyon
- Gerektiğinde kontrollü paylaşım
- Read-only bellek bölgeleri
- Fiziksel RAM’i destekleyen yardımcı mekanizmalar

Amaç

ISOLATION

PROTECTION

CONTROLLED SHARING

Virtual Memory ve MMU

Süreç fiziksel RAM’i doğrudan görmez

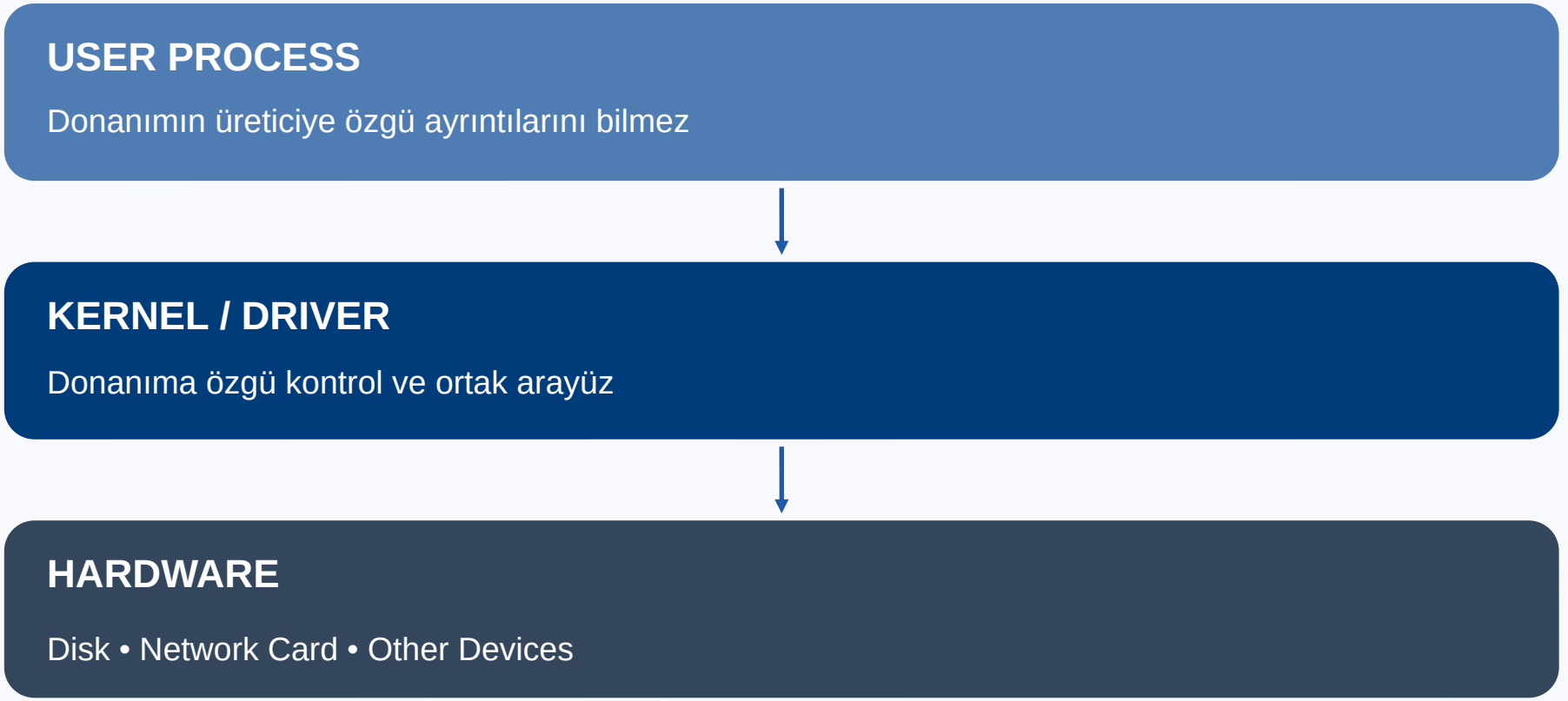


Kernel eşlemeleri ve erişim kurallarını yönetir; MMU adres çevirisini donanımsal olarak uygular.

Her süreç kendisine ait düzenli bir adres alanı varmış gibi çalışabilir.

Device Drivers

Donanım farklılıklarını kernel içinde soyutlamak



Driver = kernel ile belirli donanım arasındaki uyarlama katmanı

System Calls

User space ile kernel arasındaki kontrollü kapı



Dosya açma • okuma/yazma • süreç oluşturma • sistem kaynaklarına erişim

fork() ve exec()

Shell yeni bir programı nasıl başlatır?



fork()

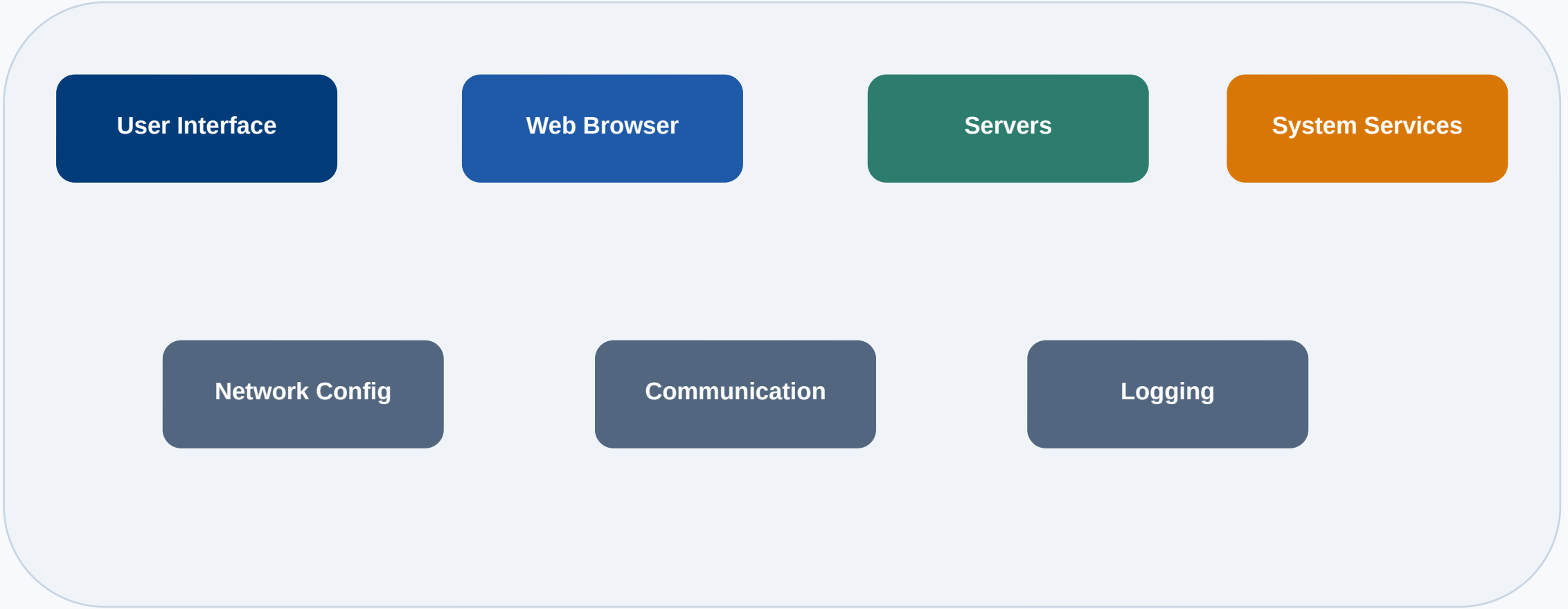
Mevcut süreçten yeni bir süreç oluşturur.

exec()

Yeni süreç oluşturmaz; mevcut sürecin çalıştırdığı programı değiştirir.

User Space

Linux’taki “asıl hareket” büyük ölçüde burada gerçekleşir



User space yalnızca insanın doğrudan kullandığı uygulamalardan oluşmaz.

User Space içinde hizmet düzeyleri

Kesin katmanlar değil, bağımlılıkları anlamaya yarayan bir model

KULLANICIYA YAKIN

User Interface • Web Browser

SERVİSLER

Web / Database / Name Services

TEMEL HİZMETLER

Network Configuration • Communication • Logging

Kernel/User Space ayrımı gerçek bir koruma sınırıdır; user space içindeki bu katmanlar kavramsaldır.

Linux’ta User

Kullanıcı adı insan için; UID kernel için

User

Süreç çalıştırabilen ve dosyalara sahip olabilen sistem varlığıdır.

Her user-space sürecinin bir sahibi vardır.

ela → UID 1001

Kernel

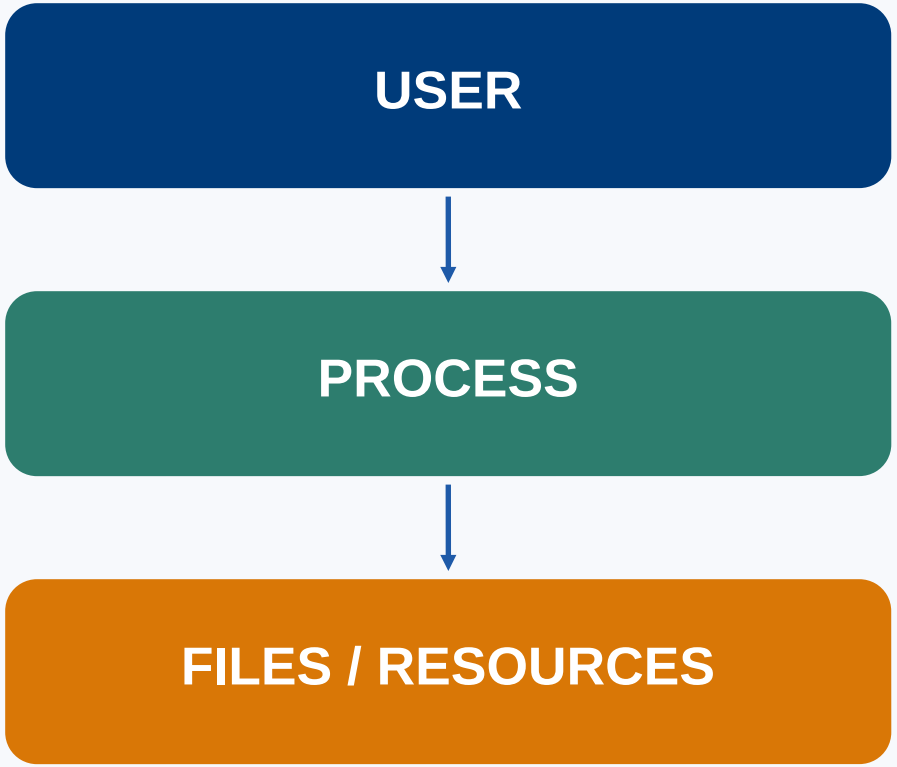
İnsan tarafından okunabilir kullanıcı adından çok sayısal kimliklerle çalışır.

Username ↔ UID eşlemesi kullanıcı yönetiminin temelidir.

User kavramı = kimlik + sahiplik + yetkilendirme

User → Process → Permissions

Kimlik, çalışan süreçlerin yetkisini belirler



Ownership + Permissions + Isolation

Süreç, çalıştığı kullanıcı kimliğinin izinleri çerçevesinde kaynaklara erişir.

Root ≠ Kernel

İki farklı ayrıcalık kavramını karıştırmamak

root / superuser

İşletim sistemi düzeyinde çok geniş kullanıcı yetkilerine sahiptir.
Dosya ve süreçler üzerinde kapsamlı yönetim işlemleri yapabilir.

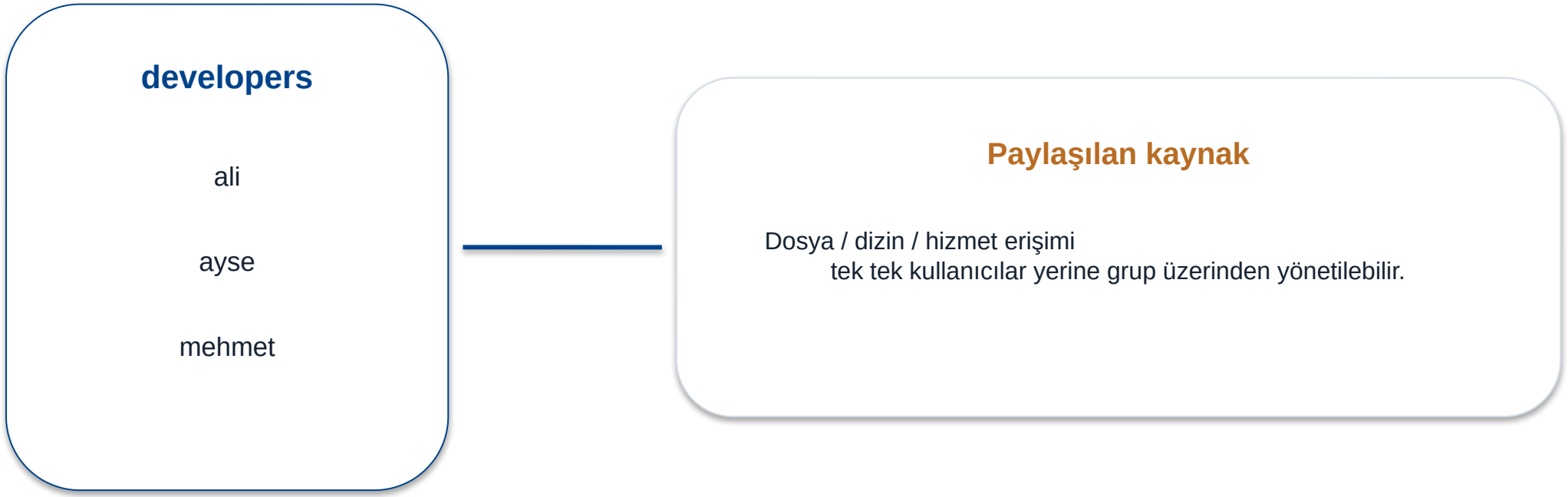
kernel mode

CPU’nun ayrıcalıklı yürütme modudur.
Kernel kritik bellek ve donanım işlemlerini burada yürütür.
Root yetkisiyle aynı mekanizma değildir.

Root tarafından başlatılan normal program yine user mode’da çalışan bir user process’tir.

Groups

Ortak erişim yetkilerini yönetmek



User + Group + Permissions

Gruplar, paylaşılan dosya ve kaynaklara ortak erişimi yönetmeyi kolaylaştırır.

Büyük resmi birleştirelim

USER / IDENTITY

UID • Groups • Permissions

USER SPACE

Applications • Shell • Services • Processes

KERNEL

Process • Memory • System Calls • Drivers

HARDWARE

CPU • RAM • Disk • Network

Linux’u anlamak = bileşenleri ezberlemek değil, sınırları ve ilişkileri görmek

Temel komutlar ve dizin hiyerarşisi

Bu bölümde

Shell kullanımı
Dosya ve dizinlerle çalışma
Metin inceleme ve arama

Büyük resimle bağlantı

Shell ve komutlar user space içinde çalışır.

Dosyalara ve diğer kaynaklara erişirken kernel hizmetlerinden yararlanırlar.

Amaç: temel komutların işlevini ve komut satırının mantığını anlamak.

Bourne Shell ve /bin/sh

Shell

Komutları alır ve yorumlar.

Yerleşik komutları veya harici programları çalıştırır.

sh ve Bash

/bin/sh
Bourne uyumlu shell arayüzü

/bin/bash
Bash (Bourne Again Shell)

/bin/sh yolunun gösterdiği shell, sisteme göre değişebilir.

Terminal penceresinde shell kullanımı

Örnek komut satırı

$ ls -l /etc

ls komut
-l seçenek
/etc argüman

Terminal ve shell

Terminal penceresi giriş ve çıktıyı gösterir.

Shell, yazdığınız komutu işler. Enter komutu gönderir.

Örneklerdeki $ prompt işaretidir. Komutun parçası olarak yazılmaz.

cat ile dosya içeriğini görüntüleme

Temel kullanım

$ cat notlar.txt

$ cat bir.txt iki.txt

Ne yapar?

Dosyaları verilen sırayla okur ve içeriği standart çıktıya yazar.

İki dosya verilirse içerikler art arda görünür.

cat, kısa metin dosyalarını incelemek için uygundur.

Standart girdi, çıktı ve hata

Üç standart akış

stdin standart girdi
stdout normal çıktı
stderr hata iletileri

Terminalde varsayılan durum

stdin genellikle terminalden veri alır.

stdout ve stderr genellikle aynı terminal ekranında görünür.

Aynı ekranda görünseler de stdout ve stderr ayrı akışlardır.

cat ile standart girdiyi gözlemleme

Terminal denemesi

$ cat
Merhaba Linux
Merhaba Linux
[Boş satırda Ctrl+D]

Girdinin sonu

cat dosya adı olmadan stdin üzerinden veri bekler.

Boş giriş satırında Ctrl+D, EOF durumunu bildirir.

Ctrl+D girdiyi bitirir. Ctrl+C genellikle çalışan programı keser.

ls ile dizin içeriğini listeleme

Temel örnekler

$ ls
$ ls /etc
$ ls -l
$ ls -F

Seçeneklerin anlamı

-l Ayrıntılı liste
Sahip, grup, boyut, zaman ve ad

-F Öğe türüne ilişkin işaretler
Örneğin dizin adının sonunda /

Argüman verilmezse ls mevcut çalışma dizinini listeler.

cp ve mv

Kopyalama, taşıma ve yeniden adlandırma

cp: kaynak yerinde kalır

```
$ cp not.txt kopya.txt

$ cp not.txt arsiv/
```

mv: konum veya ad değişir

```
$ mv not.txt ders.txt

$ mv ders.txt arsiv/
```

Genel kullanım: komut kaynak hedef

touch ve rm

Dosya oluşturma ve silme

touch

$ touch deneme.txt

Dosya yoksa boş dosya oluşturur.
Varsa içeriği değiştirmez.

rm

$ rm deneme.txt

Belirtilen dosyayı siler.
Genellikle çöp kutusuna taşımaz.

Silme örneklerini yalnızca deneme dosyalarında uygulayın.

echo ile metin yazdırma

Örnek

```
$ echo Merhaba Linux
Merhaba Linux
```

Temel davranış

echo, argümanlarını standart çıktıya yazar.

Çıktı terminale bağlıysa metin ekranda görünür.

Komutun adı echo, ardından gelen sözcükler argümanlarıdır.

Dizin hiyerarşisi ve yol kavramı

Tek bir kök dizin

/
/usr
/etc
/home
/var

Path: dosyanın konumu

/usr/lib

Kökten başlayan bir yol örneği.
Dizinleri / karakteri ayırır.

Kök dizin / ile kullanıcının ev dizini farklı konumlardır.

Mutlak ve göreli yollar

Mutlak yol

/usr/bin

/ ile başlar.
Kök dizinden tanımlanır.

Göreli yol

../bin

Mevcut çalışma dizinine göre değerlendirilir.

Çalışma dizini /usr/lib ise ../bin yolu /usr/bin konumuna gider.

Nokta ve çift nokta

Yollardaki özel dizin adları

Mevcut dizin: .

.
./notlar.txt

Mevcut dizindeki notlar.txt

Üst dizin: ..

..
../notlar.txt

Üst dizindeki notlar.txt

./notlar.txt ve notlar.txt aynı çalışma dizinindeki dosyayı belirtir.

cd ile çalışma dizinini değiştirme

Temel kullanımlar

```
$ cd /usr
$ cd ..
$ cd
```

Anlamları

cd /usr	Belirtilen dizine geçer.
cd ..	Üst dizine geçer.
cd	Ev dizinine döner.

cd, shell’in kendi çalışma dizinini değiştiren yerleşik komuttur.

mkdir ve rmdir

Dizin oluşturma ve kaldırma

mkdir

$ mkdir deneme

Yeni bir dizin oluşturur.

rmdir

$ rmdir deneme

Yalnızca boş dizini kaldırır.

rmdir, içinde dosya bulunan bir dizini kaldırmaz.

Shell globbing ve joker karakterler

Wildcards

Dosya adı kalıpları

*	Sıfır veya daha çok karakter
?	Tek karakter
[ab]	a veya b karakteri

Örnekler

```
$ ls *.txt
$ ls ders?.txt
```

Eşleşen adları komuttan önce shell genişletir.

Globbing, dosya adlarıyla eşleşir. Regex ayrı bir konudur.

grep ile metin içinde arama

Temel örnekler

```
$ grep Linux notlar.txt
$ grep -i linux notlar.txt
$ grep -v Linux notlar.txt
```

Satır seçimi

grep, kalıpla eşleşen satırları gösterir.

-i Büyük-küçük harfi ayırmaz.
-v Eşleşmeyen satırları gösterir.

Bu aşamada basit bir sözcüğü içeren satırları bulmak yeterlidir.

less ile uzun metinleri inceleme

Sayfa sayfa görüntüleme

Temel kullanım

```
$ less notlar.txt
```

Space Bir ekran ileri
b Bir ekran geri
q Çıkış

Metin arama

/Linux İleri yönde ara
?Linux Geri yönde ara

Enter ile aramayı başlatın.

less içinde metni incelersiniz, dosya içeriğini değiştirmezsiniz.

pwd ile bulunduğunuz konumu görme

Print working directory

Örnek

```
$ pwd
/home/ela/ders

$ cd ..
$ pwd
/home/ela
```

cd ve pwd

cd çalışma dizinini değiştirir.

pwd çalışma dizininin yolunu gösterir.

Göreli yolları yorumlamadan önce mevcut konumu kontrol edin.

diff ile iki metni karşılaştırma

Dosyalar arasındaki farklar

Temel kullanım

$ diff eski.txt yeni.txt

$ diff -u eski.txt yeni.txt

Çıktıyı yorumlama

diff, farklı satırları gösterir.

-u, farkları çevre satırlarla birlikte gösterir.

Aynı dosyalarda normalde çıktı oluşmaz.

diff dosyaları karşılaştırır, içeriklerini değiştirmez.

file ile dosya türünü tanıma

İçerikten tür bilgisi

Temel kullanım

```
$ file notlar.txt
notlar.txt: UTF-8 Unicode text
```

Ne inceler?

file, dosyanın içeriği ve diğer özelliklerinden tür hakkında bilgi üretir.

Yalnızca dosya uzantısına dayanmaz.

Çıktı, dosyanın gerçek içeriğine ve kullanılan sisteme göre değişir.

find ve locate ile dosya bulma

Dosya adı ve konum arama

find

$ find . -name notlar.txt -print

Mevcut dizinden başlayarak dizin ağacını tarar.

locate

$ locate notlar.txt

Önceden hazırlanmış dosya adı indeksinde arar.

locate indeksi güncel değilse yeni dosyalar sonuçta görünmeyebilir.

head ve tail

Dosyanın başını ve sonunu inceleme

head

```
$ head notlar.txt
$ head -n 5 notlar.txt
```

Varsayılan: ilk 10 satır

tail

```
$ tail notlar.txt
$ tail -n 5 notlar.txt
```

Varsayılan: son 10 satır

-n seçeneği gösterilecek satır sayısını belirler.

sort ile satırları sıralama

Metin ve sayısal sıralama

Temel örnekler

```
$ sort liste.txt
$ sort -n sayilar.txt
$ sort -r liste.txt
```

Sıralama biçimi

sort	Metin sırası
-n	Sayısal sıra
-r	Ters sıra

Sonuç standart çıktıya yazılır.

Temel kullanımda sort kaynak dosyayı değiştirmez.

Parola, shell ve dot files

2.6–2.7: Kullanıcı ortamına kısa giriş

Parola

$ passwd

passwd kullanıcı parolasını değiştirir.

Dot files

$ ls -a

Adı . ile başlayan dosya ve dizinler normal ls çıktısında görünmez.
Örnek: .bashrc, .ssh

Gizli dosyalar özel bir dosya türü değildir; yalnızca adlandırma geleneğidir.

Shell ve environment variables

2.8: Değerleri shell içinde ve süreçlere aktarmak

Shell variable

$ STUFF=blah
$ echo $STUFF
blah

Atamada = çevresinde boşluk kullanılmaz.

Environment variable

$ export STUFF

export ile değişken, shell'in başlattığı süreçlerin ortamına aktarılabilir.

Shell variable yalnızca shell bağlamındayken, export edilen environment variable child process'lere aktarılabilir.

PATH: komutlar nasıl bulunur?

2.9: Shell’in komut arama yolu

PATH değişkeni

$ echo $PATH
/usr/local/bin:/usr/bin:/bin

Dizinler : karakteriyle ayrılır.

Arama mantığı

$ ls

Shell, PATH içindeki dizinleri sırayla arar ve ilk uygun komutu çalıştırır.

$ PATH=dir:$PATH

PATH sırası önemlidir; aynı adlı programlardan önce bulunan çalıştırılır.

Komut satırını hızlı düzenleme

2.11: Terminalde temel klavye hareketleri

Gezinme

Ctrl+A Satır başı
Ctrl+E Satır sonu
Ctrl+P Önceki komut
Ctrl+N Sonraki komut

Düzenleme

Ctrl+W Önceki sözcüğü sil
Ctrl+U İmleçten satır başına sil
Ctrl+K İmleçten satır sonuna sil
Ctrl+Y Silinen metni geri ekle

Ok tuşları kullanılabilir; bu kısayollar birçok Unix aracında da karşınıza çıkar.

Metin editörleri

2.12: Yapılandırma dosyalarını güvenli biçimde düzenlemek

Terminal editörleri

vi / vim
nano

Terminalin standart giriş/çıkış ortamında çalışırlar.

Diğer seçenekler

Emacs
GUI editörleri

Editör seçimi iş akışına göre değişebilir.

Linux yönetiminde en az bir terminal tabanlı editörü temel düzeyde kullanabilmek önemlidir.

Linux’ta çevrimiçi yardım

2.13: man, apropos ve info

man pages

$ man ls
$ man -k sort
$ man 5 passwd

Komut veya konu için sistemdeki referans belgelerini açar.

Diğer yardım yolları

$ command --help
$ info command

Paket belgeleri /usr/share/doc altında da bulunabilir.

man sayfaları genellikle öğretici metinden çok başvuru kaynağıdır.

Shell input/output yönlendirme

2.14: Çıktıyı terminal dışına taşımak

Dosyaya yönlendirme

$ command > file
$ command >> file

> üzerine yazar.
>> sona ekler.

Pipe

$ head /proc/cpuinfo | tr a-z A-Z

| bir komutun stdout'unu sonraki komutun stdin'ine bağlar.

Komutları küçük parçalar halinde birleştirmek Unix yaklaşımının temel özelliklerinden biridir.

Unix hata mesajını okumak

2.15: Önce ilk hatayı anlayın

Örnek

```
$ ls /dsafsda
ls: cannot access /dsafsda:
No such file or directory
```

Üç parça

Program: ls
Hedef: /dsafsda
Hata: No such file or directory

İlk hatayı önce çözün.

Unix hata mesajları çoğu zaman problemin kaynağını doğrudan söyler.

Sık karşılaşılan hatalar

2.15.2: Mesajdan probleme gitmek

Dosya ve erişim

No such file or directory
File exists
Not a directory / Is a directory
Permission denied

Kaynak ve süreç

No space left on device
Operation not permitted
Segmentation fault
Bus error

Hata mesajı semptomdur; doğru çözüm için hangi kaynak ve işlemin başarısız olduğunu belirleyin.

Süreçleri listelemek: ps

2.16: Çalışan programları gözlemlemek

Temel kullanım

```
$ ps
$ ps x
$ ps ax
$ ps u
$ ps w
```

Önemli alanlar

PID Process ID
TTY Terminal
STAT Süreç durumu
TIME Kullanılan CPU zamanı
COMMAND Komut

PID çalışan süreç için sayısal kimliktir; süreç sonlandığında daha sonra yeniden kullanılabilir.

Süreçleri sonlandırmak: signals

2.16.2: Sürece kernel üzerinden mesaj göndermek

Temel sinyaller

```
$ kill PID       # TERM
$ kill -STOP PID
$ kill -CONT PID
```

Zorunlu sonlandırma

```
$ kill -KILL PID
```

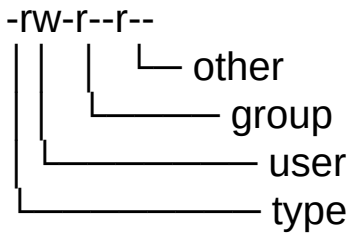
KILL yakalanamaz veya yok sayılamaz.
Ctrl+C genellikle INT sinyali gönderir.

Önce normal sonlandırmayı deneyin; KILL son çare olarak düşünülmelidir.

Dosya modları ve izinler

2.17: user, group ve other

Örnek mod



İzin karakterleri

r read
w write
x execute
- izin yok

Üç izin seti: user • group • other

Dosya türü ile erişim izinleri aynı mode alanında birlikte gösterilir.

chmod ile izin değiştirme

2.17.1: Sembolik ve sayısal gösterim

Sembolik örnekler

$ chmod g+r file
$ chmod o-r file
$ chmod go+r file

Sayısal örnek

$ chmod 644 file

4 = read
2 = write
1 = execute

644 → rw-r--r--

Yetki vermeden önce hangi kullanıcı veya grubun gerçekten erişime ihtiyacı olduğunu düşünün.

Sembolik bağlantılar

2.17.2: Bir ada başka bir yolu işaret ettirmek

Oluşturma

$ ln -s target linkname

linkname → target

Davranış

Symlink hedef yolun kendisini içerir.
Hedef taşınır veya silinirse bağlantı bozulabilir.

$ ls -l

Sembolik bağlantı dosyanın kopyası değildir; başka bir ada/yola referanstır.

Arşivleme ve sıkıştırma

2.18: tar ve gzip farklı işler yapar

gzip

$ gzip file
$ gunzip file.gz

Tek bir veri akışını sıkıştırır.

tar

$ tar cvf archive.tar file1 file2
$ tar xvf archive.tar

Birden çok dosya ve dizini tek arşivde toplar.

tar arşivler; gzip sıkıştırır. Birlikte kullanıldıklarında .tar.gz / .tgz biçimi yaygındır.

Sıkıştırılmış tar arşivleri

2.18.3–2.18.4: .tar.gz ve akış yaklaşımı

Yaygın kullanım

$ tar czvf archive.tar.gz dir/
$ tar xzvf archive.tar.gz

(z seçeneği gzip sıkıştırmasını ekler.)

Akış örneği

$ zcat file.tar.gz | tar xvf -

- bazı Unix araçlarında stdin/stdout anlamında kullanılabilir.

.tar.gz tek bir “özel dosya türü”nden çok arşivleme + sıkıştırma adımlarının birleşimidir.

Linux dizin hiyerarşisi

2.19: Kök dizin altındaki temel roller

Sistem ve yazılımlar

/bin temel çalıştırılabilirler
/etc sistem yapılandırması
/usr kullanıcı alanı yazılım/verileri
/lib paylaşılan kütüphaneler
/sbin sistem araçları

Çalışma ve veri

/home kullanıcı ev dizinleri
/var değişken veriler/loglar
/tmp geçici dosyalar
/dev aygıt düğümleri
/proc süreç/kernel görünümü

Dizinlerin anlamını ezberlemekten çok, bir dosyanın neden o bölgede bulunabileceğini anlamaya odaklanın.

Dizin hiyerarşisinde birkaç kritik nokta

2.19: /run, /boot ve /opt

Çalışma zamanı

/run boot sonrası çalışma zamanı verileri
/boot kernel ve boot loader dosyaları

/run çoğunlukla yeniden başlatmada yeniden oluşturulur.

Ek yazılımlar

/opt ek/üçüncü taraf yazılımlar için kullanılabilir

/usr/local da yerel kurulumlarda sık görülür.

Linux dizin ağacı tek kökten başlar; farklı depolama aygıtları bu ağacın belirli noktalarına bağlanabilir.

Komutları superuser yetkisiyle çalıştırmak

2.20: root shell yerine kontrollü yükseltme

sudo

$ sudo vipw
$ sudo command

Yetkili kullanıcıya belirli komutu yükseltilmiş yetkiyle çalıştırma imkânı verir.

Neden tercih edilir?

İşlem kaydı tutulabilir.
Normal kullanıcı ortamında çalışmaya devam edilir.
Gereksiz sürekli root oturumu azaltılır.

Root yetkisi güçlüdür; yalnızca gerçekten gereken işlem için yükseltmek daha güvenli bir çalışma alışkanlığıdır.

sudoers ve sudo kayıtları

2.20.2–2.20.3: Yetki politikası ve izlenebilirlik

/etc/sudoers

user1 ALL = NOPASSWD: ALL

Politika hangi kullanıcının hangi yetkiyle hangi komutları çalıştırabileceğini belirler.

Düzenleme için: visudo

Kayıtlar

$ journalctl SYSLOG_IDENTIFIER=sudo

Sisteme göre sudo olayları journal veya log dosyalarında tutulabilir.

sudoers dosyasını doğrudan rastgele düzenlemek yerine visudo kullanmak sözdizimi hatası riskini azaltır.

LINUX TEMELLERİ

Büyük Resim: Kernel, Süreçler, Bellek ve User Space

Öğr. Gör. Dr. Uğur ÖZBEK

SAKARYA ÜNİVERSİTESİ